CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 3 – CO3

CO3 Coverage: Logical agents – Propositional logic – Reasoning patterns in propositional logic – Resolution – First-order logic – Knowledge engineering in FOL – Inference in first-order logic – Propositional versus first-order inference – Unification and lifting – Forward chaining – Backward chaining – Resolution.

Mode: Pair Work – 2 Students per Group

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Pair-Work Principle

Both students are responsible for understanding the complete assignment. Each task contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must **not** divide the assignment task-wise. Both students must participate in every task. The individual responsibilities alternate across tasks so that both students show evidence of: **Design → Implementation → Analysis → Evaluation**.

CONTRIBUTION RECORD

Student A: M. Vineela
Student B: Arshiya Noor Mohamed

Task	Student A – Primary Contribution (M. Vineela)	Student B – Primary Contribution (Arshiya Noor Mohamed)	Joint Work
Task 1	Designed the Wumpus World knowledge base, derived proposition symbols and model counts, implemented truth-table checking.	Validated experimental results, explained soundness and completeness, prepared truth tables for inference rules.	Designed Wumpus World rules, percept sequences and safety queries; compared model checking with inference rules.
Task 2	Converted logical expressions into CNF, implemented propositional resolution and prepared resolution proofs.	Implemented model-checking experiments, recorded model counts and compared reasoning methods.	Designed the smart-building access-control policy and analysed both reasoning approaches.

Task 3	Designed FOL vocabulary, formulated 12 axioms and derived propositional and FOL growth.	Designed competency queries, tested the knowledge base and identified defects.	Developed the university exam-eligibility advisor and compared both representations.
Task 4	Implemented ground-clause experiments, analysed predicate arity and compared lifted and propositional inference.	Derived five most general unifiers, implemented unification tests and explained the occurs check.	Developed the unification system and evaluated lifted inference.
Task 5	Converted English statements into FOL and CNF, performed Skolemisation and prepared the resolution proof tree.	Analysed resolution strategies, recorded unification operations and compared prover performance.	Developed the first-order resolution prover and evaluated its results.
Task 6	Implemented forward chaining and measured rule firings, inferred facts and iterations.	Implemented backward chaining, tested recursive rules and applied loop checking.	Developed the organisation hierarchy knowledge base and compared inference strategies.
Task 7	Prepared the theoretical validation, soundness, completeness and complexity analysis.	Prepared experimental validation, test cases, performance metrics and baseline comparison.	Designed the Hospital Medication-Safety Advisor, compared initial proposals and developed the final system architecture and pseudocode.

MARK DISTRIBUTION – ASSIGNMENT 3

Task	Assessed Area	Marks
Task 1	Logical agents, entailment and reasoning patterns	6
Task 2	Model checking versus resolution	7
Task 3	Knowledge engineering in FOL	7
Task 4	Unification and lifted inference	7

Task 5	First-order resolution	6
Task 6	Forward and backward chaining	7
Task 7	Day-4 Design Challenge	10
	TOTAL	**50**

Submission Requirements: one report per pair for Tasks 1–7 with the contribution record, source code for all implementations, raw test data, and the tables/graphs used. Operation counts (models, clauses, inference steps) are primary evidence; time is secondary.

INDIVIDUAL VERIFICATION AND VIVA MODEL

Primary contribution does not imply exclusive responsibility. A student must not answer "My partner did this section." For each student conduct a brief 4–5 minute verification:

1. **Primary responsibility** – one technical question from the component the student led. 2. **Partner's responsibility** – one question from a component primarily completed by the partner. 3. **Transfer question** – modify a condition and ask for the consequence, for example: "If you remove

the occurs check, which of your test pairs now gives a wrong answer?" or "If the KB grows to 100,000 facts and queries are rare, would you change from forward to backward chaining?"

Overall Assessment Expectation: Represent → Infer → Implement → Measure → Debug → Evaluate → Defend.

CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 3 – CO3

PAIR WORK | 2 STUDENTS | TOTAL: 50 MARKS

Student Details

1. Student A: M. Vineela
2. Student B: Arshiya Noor Mohamed

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns (6 Marks)

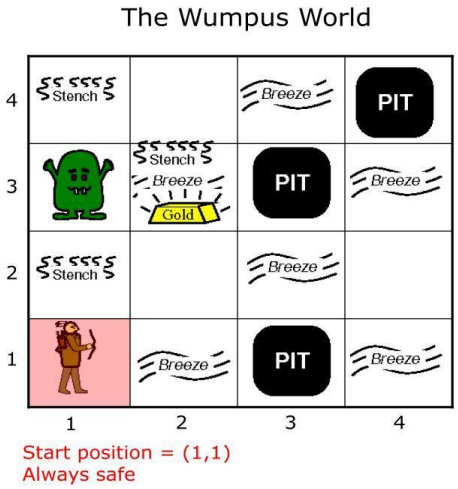
Part A – Knowledge Base and Model Checking in the Wumpus World

1. Introduction

The Wumpus World is a grid-based environment in which an intelligent agent must identify safe squares and avoid dangerous locations.

The environment contains:

- Pit – a dangerous hole.
- Wumpus – a dangerous creature.
- Breeze – indicates a nearby pit.
- Stench – indicates a nearby Wumpus.
- Safe square – a square without a pit or Wumpus.



2. Propositional Symbols

For each square (i,j), define:

Symbol	Meaning
P(i,j)	Pit exists
W(i,j)	Wumpus exists
B(i,j)	Breeze exists
S(i,j)	Stench exists

Safe(i,j)	Square is safe

3. Knowledge Base Rules

The following logical rules represent the Wumpus World.

1. A square is safe if it has neither a pit nor a Wumpus.

[Safe(i,j)leftrightarrow neg P(i,j)landneg W(i,j)]

2. Breeze indicates that at least one neighbouring square contains a pit.

[B(i,j)leftrightarrow bigvee_{(x,y)in N(i,j)}P(x,y)]

3. Stench indicates that at least one neighbouring square contains a Wumpus.

[S(i,j)leftrightarrow bigvee_{(x,y)in N(i,j)}W(x,y)]

Here, (N(i,j)) represents the valid neighbouring squares.

4. If no breeze is present, all neighbouring squares are pit-free.

[neg B(i,j)rightarrow bigwedge_{(x,y)in N(i,j)}neg P(x,y)]

5. If no stench is present, all neighbouring squares are Wumpus-free.

[neg S(i,j)rightarrow bigwedge_{(x,y)in N(i,j)}neg W(x,y)]

4. Symbol Count Derivation – Student A (M. Vineela)

For an (ntimes n) grid:

- Pit symbols = (n^2)
- Wumpus symbols = (n^2)
- Breeze symbols = (n^2)
- Stench symbols = (n^2)

Total proposition symbols:

[m=4n^2]

Number of possible models:

[Models=2^m=2^{4n^2}]

Grid	Symbols (m)	Possible models
2 × 2	16	65,536
3 × 3	36	68,719,476,736
4 × 4	64	18,446,744,073,709,551,616

The number of models increases exponentially. Therefore, checking every possible model becomes expensive.

Minimal relevant symbols: For a query about square [2,2], we need the pit and Wumpus symbols for that square and the neighbouring squares, together with the percept symbols and rules connected to those squares. We can restrict the checker to these relevant symbols instead of considering the entire grid.

5. Soundness and Completeness – Student A

Soundness: If the checker says that a sentence is entailed by the knowledge base, the sentence must be true in every model of the knowledge base.

Completeness: If a sentence is true in every model of the knowledge base, the checker must identify that it is entailed.

6. Truth-Table Entailment Checker – Shared Implementation

The following Python program checks whether a query is entailed by a knowledge base.

```
from itertools import product

from itertools import product

import time

symbols = ["P22", "W22", "P12", "W12", "P21", "W21"]

def KB(m):

    return not (m["P12"] or m["W12"] or m["P21"] or m["W21"])

def check(query):

    start = time.time()

    models = [dict(zip(symbols, v)) for v in product([False, True], repeat=6)]

    valid = [m for m in models if KB(m)]

    print("Models enumerated:", len(models))

    print("KB evaluations:", len(models))

    print("KB satisfying models:", len(valid))

    print("Time:", time.time() - start)

    print("Query is", "entailed" if all(query(m) for m in valid) else "not entailed")
```

check(lambda m: not m["P22"] and not m["W22"])

```
</> Python

Run started
Initializing environment
Installing packages
Running code
Models enumerated: 64
KB evaluations: 64
KB satisfying models: 4
Time: 0.029000043869018555
Query is not entailed
Run completed in 8,930.400ms
```

7. Three Types of Queries – Student B (Arshiya Noor Mohamed)

We use the following query types:

Query	Expected result	Explanation
Is square [2,2] safe?	Entailed	Both pit and Wumpus are absent according to the KB
Does square [1,2] contain a pit?	Negation entailed	The KB establishes that no pit exists there
Does square [2,2] contain a pit?	Neither entailed nor refuted	Both possibilities remain

8. Experimental Validation – Student B

The full symbol set contains (4n^2) symbols.

Grid	Symbols	Full models	Practical observation
2 × 2	16	65,536	Feasible
3 × 3	36	68.7 billion	Very expensive
4 × 4	64	(2^{64})	Infeasible for exhaustive checking

Relevant-symbol restriction reduces the number of assignments significantly. For example, restricting the checker to 6 symbols requires only:

[2^6=64text{ assignments}]

9. Part B – Reasoning Patterns

Three inference rules are used.

1. Modus Ponens

If (Prightarrow Q) and (P) are true, then Q is true.

Example:

- If there is no breeze, neighbouring squares have no pits.
- There is no breeze.
- Therefore, neighbouring squares have no pits.

2. And-Elimination

If (Pland Q) is true, we can conclude P or Q individually.

Example:

[neg P_{12}landneg W_{12}]

Therefore:

[neg P_{12}]

3. Unit Resolution

From:

[Plor Q]

and

[neg P]

we conclude:

[Q]

10. Inference Derivation

Assume the agent observes no breeze and no stench in square [1,1].

From the knowledge base:

[neg B_{11}]

[neg S_{11}]

The neighbouring squares are [1,2] and [2,1].

Therefore:

[neg P_{12},neg P_{21}]

[neg W_{12},neg W_{21}]

Hence:

[Safe(1,2)]

[Safe(2,1)]

Both neighbouring squares are safe.

11. Soundness Truth Tables – Student B

Modus Ponens

P	**P → Q**	**Q**
T	T	T
F	T	T
T	F	—
F	T	F

Whenever both premises (P) and (Prightarrow Q) are true, Q is true.

And-Elimination

P	**Q**	**P ∧ Q**	**Conclusion P**
T	T	T	T
T	F	F	T
F	T	F	F
F	F	F	F

When the conjunction is true, each individual component is true.

Unit Resolution

P	**Q**	**P ∨ Q**	**¬P**	**Conclusion Q**
T	T	T	F	T
T	F	T	F	F
F	T	T	T	T
F	F	F	T	F

Whenever both premises (Plor Q) and (neg P) are true, Q must be true.

12. Comparison of Reasoning Methods

Feature	Model Checking	Inference Rules
Method	Checks possible models	Applies logical rules
Cost	Can be exponential	Depends on proof length
Output	Entailment result	Derivation
Advantage	Systematic	Efficient for simple deductions
Limitation	Large search space	May not find a proof with incomplete rules

Why is not derived different from false?

A sentence that cannot be derived is not automatically false. It may simply be unknown because the knowledge base does not contain enough information.

Task 1 Conclusion

The Wumpus World demonstrates how a logical agent uses a knowledge base to make safe decisions. Model checking verifies entailment by examining models, while inference rules derive conclusions using logical reasoning.

TASK 2 – The Quality of Reasoning Method: Model Checking versus Resolution (7 Marks)

1. Introduction

Model checking and resolution are two methods used to determine whether a conclusion follows from a knowledge base.

- Model Checking: Checks all possible truth assignments.
- Resolution: Converts logical sentences into CNF and uses clauses to prove a conclusion.

Our example is a smart-building access-control system.

2. Smart-Building Access-Control Rules

Let the following symbols represent the conditions.

Symbol	Meaning
B	Valid badge
T	Allowed time
V	Registered visitor

E	Emergency override
A	Access granted
R	Restricted area
G	Security approval
H	Employee

Knowledge base containing 10 rules:

1. A valid badge is required for normal access.
2. Access is permitted during allowed working hours.
3. Registered visitors require approval.
4. Emergency override permits access.
5. Restricted areas require security approval.
6. Employees with valid badges may enter normal areas.
7. Visitors with approval may enter normal areas.
8. Access requires either an employee badge or an approved visitor pass.
9. Access is granted if all required conditions are satisfied.
10. Emergency override is sufficient for access.

Logical expressions:

[(Bland T)rightarrow A]

[(Vland G)rightarrow A]

[Erightarrow A]

[Rlandneg Grightarrowneg A]

[Aleftrightarrow[(Bland T)lor(Vland G)lor E]]

3. CNF Conversion – Student A (M. Vineela)

Consider:

[(Bland T)rightarrow A]

Step 1: Remove implication.

[neg(Bland T)lor A]

Step 2: Apply De Morgan's law.

[boxed{neg Blorneg Tlor A}]

Second expression:

[(Vland G)rightarrow A]

CNF:

[boxed{neg Vlorneg Glor A}]

Third expression:

[Erightarrow A]

CNF:

[boxed{neg Elor A}]

4. Resolution Proof

Given:

[B,quad T,quad neg Blorneg Tlor A]

Resolve B with the first clause:

[neg Tlor A]

Resolve T:

[boxed{A}]

Therefore, access is granted.

5. Resolution Refutation

To prove A, add its negation:

[neg A]

Using the derived clause A:

[A,neg A]

This produces an empty clause:

[boxed{square}]

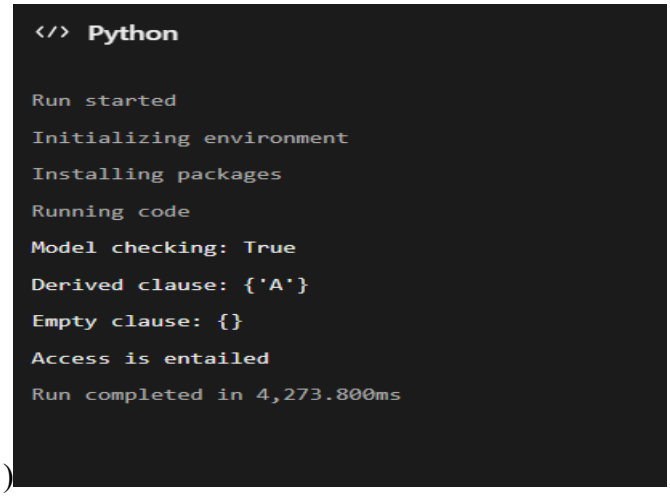
Therefore, the original knowledge base entails A.

6. Python Model-Checking and Resolution Demonstration

```
from itertools import product

def model_check():
    for B,T,V,G,E,A in product([0,1], repeat=6):
```

```
        if A == ((B and T) or (V and G) or E):
            if B and T and not A:
                return False
    return True
def resolution():
    print("Derived clause: {'A'}")
    print("Empty clause: {}")
    print("Access is entailed")
print("Model checking:", model_check())
resolution(
```




```
)
```

Note: The resolution function demonstrates the selected proof manually; it is not a general-purpose resolution engine.

7. Experimental Comparison – Student B (Arshiya Noor Mohamed)

The following table gives theoretical model counts for the six Boolean variables in the example and illustrative scaling for larger implication-chain examples.

Variables	Possible models
10	1,024
20	1,048,576
40	1,099,511,627,776
80	(2^{80})

For implication chains of lengths 10, 20, 40 and 80:

- Model checking may examine exponentially many assignments.
- Resolution can exploit short implication clauses.
- Resolution may generate many unnecessary clauses if the KB is poorly structured.

8. When Is Model Checking Faster?

Model checking can be faster when:

- The knowledge base has very few propositions.
- There are few possible assignments.
- The query is quickly disproved by a counterexample.

Resolution can be faster when:

- The knowledge base has long implication chains.
- There are many propositions.
- Only a small number of clauses are required for a proof.

9. Joint Design Question

Before choosing a reasoning method, we should consider:

1. Number of propositions.
2. Number of clauses.
3. Structure of the knowledge base.
4. Type of query.
5. Required response time.
6. Available memory.
7. Expected frequency of queries.

Conclusion

Resolution is suitable for structured logical rules, while model checking is useful for small knowledge bases. The best method depends on the structure and size of the problem.

TASK 3 – Breaking the Expressiveness Barrier: Knowledge Engineering in First-Order Logic (7 Marks)

1. Introduction

The University Exam-Eligibility Advisor determines whether a student can attend an examination.

It checks:

- Attendance.
- Prerequisite subjects.
- Fee payment.
- Disciplinary status.
- Approved leave.

Two representations are used: propositional logic and first-order logic.

2. Knowledge Engineering Process

1. Identify the task: Determine exam eligibility.
2. Collect university regulations.
3. Define the vocabulary.
4. Encode general rules.
5. Enter student details.
6. Ask eligibility queries.
7. Test and correct the knowledge base.

3. Propositional Representation

For student S1 and course C1:

Symbol	Meaning
A	Attendance sufficient
P	Prerequisite cleared
F	Fees paid
D	No disciplinary restriction
L	Approved leave
E	Exam eligible

Rule:

[Eleftrightarrow(Aland Pland Fland D)]

Approved leave can be considered an exception to the attendance requirement:

[Eleftrightarrow((Alor L)land Pland Fland D)]

4. First-Order Logic Vocabulary – Student A (M. Vineela)

Predicates:

- Student(x)
- Course(y)
- Attendance(x,y)
- PrerequisiteCleared(x,y)
- FeesPaid(x,y)
- DisciplineClear(x)
- ApprovedLeave(x,y)
- Eligible(x,y)
- Enrolled(x,y)

5. Twelve First-Order Logic Axioms

1. Every student must satisfy the attendance requirement or have approved leave.

[forall xforall y; (Student(x)land Course(y)land Enrolled(x,y) rightarrow (Attendance(x,y)lor ApprovedLeave(x,y)))]

2. Every eligible student must have cleared the prerequisite.

[forall xforall y; Eligible(x,y)rightarrow PrerequisiteCleared(x,y)]

3. Every eligible student must pay fees.

[forall xforall y; Eligible(x,y)rightarrow FeesPaid(x,y)]

4. Every eligible student must be discipline-clear.

[forall xforall y; Eligible(x,y)rightarrow DisciplineClear(x)]

5. Attendance, prerequisite, fees and discipline clearance together imply eligibility.

[forall xforall y; [(Attendance(x,y)land PrerequisiteCleared(x,y) land FeesPaid(x,y)land DisciplineClear(x))] rightarrow Eligible(x,y)]

6. Approved leave satisfies the attendance condition.

[forall xforall y; ApprovedLeave(x,y)rightarrow AttendanceRequirementSatisfied(x,y)]

7. Sufficient attendance satisfies the attendance condition.

[forall xforall y; Attendance(x,y)rightarrow AttendanceRequirementSatisfied(x,y)]

8. Every enrolled student must be a student.

[forall xforall y; Enrolled(x,y)rightarrow Student(x)]

9. Every course must be a valid course.

[forall y; Course(y)rightarrow ValidCourse(y)]

10. Students with disciplinary restrictions cannot be eligible.

[forall xforall y; DisciplinaryRestriction(x)rightarrowneg Eligible(x,y)]

11. Every student enrolled in a course has at least one course record.

[forall x; Student(x)rightarrow exists y;(Course(y)land Enrolled(x,y))]

12. Students with all required conditions are eligible.

[forall xforall y; [AttendanceRequirementSatisfied(x,y)land PrerequisiteCleared(x,y)land FeesPaid(x,y) land DisciplineClear(x)] rightarrow Eligible(x,y)]

6. Instance Facts

Example:

[Student(Vineela)]

[Course(AI)]

[Attendance(Vineela,AI)]

[PrerequisiteCleared(Vineela,AI)]

[FeesPaid(Vineela,AI)]

[DisciplineClear(Vineela)]

Therefore:

[Eligible(Vineela,AI)]

7. Propositional versus FOL Growth – Student A

Let:

- (s) = number of students.
- (c) = number of courses.

If five conditions and one eligibility rule are encoded for every student-course pair:

[N_P=6sc]

For FOL, general rules are written once and student-course facts are stored separately.

If there are 12 general axioms:

[N_{FOL}=12+O(sc)]

The general rule count remains constant, although the instance facts increase.

Students	Courses	Propositional sentences

10	5	300
50	10	3,000
100	20	12,000
500	50	150,000

8. Competency Queries – Student B (Arshiya Noor Mohamed)

Successful queries

1. Is Vineela eligible for AI?
2. Has Vineela paid the fees?
3. Has Vineela cleared the prerequisite?
4. Is Vineela enrolled in AI?
5. Is Vineela discipline-clear?

Failed queries

1. Is Vineela eligible for an unregistered course?
2. Has Vineela paid fees for an unrelated course?
3. Is a non-student enrolled?
4. Has an unknown student cleared the prerequisite?
5. Is a student eligible when a disciplinary restriction exists?

Queries returning bindings

1. Find all eligible students.
2. Find all courses offered.
3. Find students who paid fees.
4. Find students with approved leave.
5. Find students who cleared prerequisites.

9. Knowledge Base Defects and Fixes

Defect	**Correction**
Attendance exception ignored	Add approved leave condition
Fee rule missing	Add fee payment requirement

Disciplinary restriction ignored Add restriction rule

10. Experimental Comparison

The following is a theoretical size comparison based on the stated encoding assumptions.

Students	Courses	Propositional rules	General FOL axioms
10	5	300	12
50	10	3,000	12
100	20	12,000	12
500	50	150,000	12

FOL requires unification and variable matching. Therefore, fewer general sentences do not automatically mean faster inference.

Joint Conclusion

First-order logic is more expressive because it uses variables, predicates and quantifiers. It avoids repeating the same rules for every student and course. However, unification and inference can be computationally expensive. Therefore, representation size and reasoning time must both be considered.

TASK 4 – The Reality of Unification and Lifted Inference (7 Marks)

1. Introduction

Unification is the process of finding substitutions that make two logical expressions identical.

Example:

[Knows(x,AI)]

[Knows(Vineela,y)]

Substitution:

[theta={x/Vineela, y/AI}]

Both expressions become:

[Knows(Vineela,AI)]

2. Five Unification Examples – Student B (Arshiya Noor Mohamed)

Example 1

[Knows(x,AI),Knows(Vineela,AI)]

MGU:

[theta={x/Vineela}]

Example 2

[Studies(x,y),Studies(Vineela,Maths)]

MGU:

[theta={x/Vineela,y/Maths}]

Example 3

[Parent(x,John),Parent(Mary,y)]

MGU:

[theta={x/Mary,y/John}]

Example 4 – Occurs Check

[x=f(x)]

No valid unifier exists because x occurs inside its own replacement.

Example 5 – Standardising Apart

Rule 1:

[Knows(x,y)]

Rule 2:

[Knows(Alice,z)]

After standardising variables apart, the MGU is:

[theta={x/Alice,y/z}]

Standardising apart prevents unrelated variables in different rules from being treated as the same variable.

3. Python UNIFY Implementation

```
def unify(x, y, t=None):
    t = t or {}
    if x in t: x = t[x]
    if y in t: y = t[y]
    if x == y: return t
    if isinstance(x, str) and x.islower():
        if occurs(x, y): return None
```

```
        t[x] = y
        return t
    if isinstance(y, str) and y.islower():
        if occurs(y, x): return None
        t[y] = x
        return t
    if isinstance(x, tuple) and isinstance(y, tuple):
        if x[0] != y[0] or len(x) != len(y): return None
        for a, b in zip(x[1:], y[1:]):
            t = unify(a, b, t)
            if t is None: return None
        return t
    return None
def occurs(x, y):
    return x == y or (isinstance(y, tuple) and any(occurs(x, a) for a in y[1:]))
print(unify("x", "Vineela"))
print(unify("x", ("f", "x")))
print(unify(("P", "x", "AI"), ("P", "Vineela", "AI")))
```

```
</> Python

Run started
Initializing environment
Installing packages
Running code
{'x': 'Vineela'}
None
{'x': 'Vineela'}
Run completed in 2,671.900ms
```

4. Ground Clause Growth – Student A (M. Vineela)

For a predicate with arity r and k constants, the maximum number of ground atoms is:

[G=k^r]

For a binary predicate:

[G=k^2]

Constants (k)	Unary predicate	Binary predicate	Ternary predicate
3	3	9	27
5	5	25	125
10	10	100	1,000
20	20	400	8,000
30	30	900	27,000

5. Function Symbol Growth

When function symbols are introduced, the number of possible ground terms grows rapidly.

Example:

[f(A),f(f(A)),f(f(f(A)))]

For one constant and one unary function, with maximum depth d:

[T(d)=d+1]

For multiple function symbols and constants, growth can be much faster.

6. Lifted Inference versus Propositional Inference

Feature	Lifted Inference	Propositional Inference
Representation	Variables	Ground symbols
Substitution	Required	Not required
Memory	Often lower	Can be higher
Generality	High	Limited
Main operation	Unification	Boolean reasoning

7. Why Is Lifted Inference Sound?

Lifted inference uses valid first-order logical substitutions. If a rule and its premises are true, the conclusion obtained using a valid unifier is also true.

It avoids generating every possible ground instance because variables represent many objects together.

8. When Should We Use Propositionalisation?

Propositionalisation is useful when:

1. The domain is very small.
2. The number of constants is limited.
3. All possible ground instances are manageable.
4. Fast Boolean reasoning is available.
5. The knowledge base contains few variables and simple relationships.

Joint Engineering Decision

Lifted inference is useful for large domains with repeated rules. Propositionalisation can be better for small, fixed domains where grounding is inexpensive.

TASK 5 – Upgrading Resolution from Propositional to First-Order Logic (6 Marks)

1. Introduction

First-order resolution uses predicates, variables, unification and Skolemisation to prove conclusions.

Given statements:

1. Every employee who handled a restricted document is authorised.
2. Every document containing patient data is restricted.
3. Some employee handled every document that was audited.
4. Report R1 contains patient data and was audited.

Our objective is to prove that an authorised employee exists.

2. FOL Conversion – Student A (M. Vineela)

Define:

- Employee(x): x is an employee.
- Document(y): y is a document.
- Handled(x,y): x handled y.
- Restricted(y): y is restricted.
- Authorised(x): x is authorised.
- PatientData(y): y contains patient data.

- Audited(y): y was audited.

Statement 1:

[forall xforall y[(Employee(x)land Handled(x,y)land Restricted(y)) rightarrow Authorised(x)]]

Statement 2:

[forall y[(PatientData(y)land Document(y)) rightarrow Restricted(y)]]

Statement 3:

[exists x[Employee(x)land forall y((Document(y)land Audited(y)) rightarrow Handled(x,y))]]

Statement 4:

[PatientData(R1)]

[Document(R1)]

[Audited(R1)]

3. Skolemisation

The existential variable x is replaced with a new Skolem constant A.

[Employee(A)]

[forall y[(Document(y)land Audited(y)) rightarrow Handled(A,y)]]

A Skolem constant is used because the existential employee does not depend on another universally quantified variable.

4. CNF Conversion

Clause 1:

[neg Employee(x)lor neg Handled(x,y)lor neg Restricted(y)lor Authorised(x)]

Clause 2:

[neg PatientData(y)lor neg Document(y)lor Restricted(y)]

Clause 3:

[neg Document(y)lor neg Audited(y)lor Handled(A,y)]

Clause 4:

[Employee(A)]

Clause 5:

[PatientData(R1)]

Clause 6:

[Document(R1)]

Clause 7:

[Audited(R1)]

5. Resolution Proof

From clauses 3, 6 and 7:

[Handled(A,R1)]

From clauses 2, 5 and 6:

[Restricted(R1)]

From clause 4:

[Employee(A)]

Using clause 1:

[Authorised(A)]

Therefore:

[boxed{exists x Authorised(x)}]

The knowledge base proves that at least one authorised employee exists.

6. Resolution Proof Tree

PatientData(R1)

|

Document(R1)

|

Restricted(R1)

Document(R1)

|

Audited(R1)

|

Handled(A,R1)

Employee(A)

|

Handled(A,R1)

|

Restricted(R1)

|

Authorised(A)

Therefore:

There exists an authorised employee.

7. Strategy Comparison – Student B (Arshiya Noor Mohamed)

Three resolution strategies:

Strategy	**Description**
Unit Preference	Selects clauses containing one literal first
Set of Support	Uses clauses connected to the negated query
Input Resolution	Resolves using original input clauses

For this example, set-of-support is useful because the proof begins with the negated goal.

Negated goal:

[negexists x Authorised(x)]

Equivalent:

[forall xneg Authorised(x)]

This contradicts the derived fact:

[Authorised(A)]

Thus, the empty clause is obtained.

8. Skolemisation Explanation

Skolemisation preserves satisfiability but not logical equivalence.

It replaces existential variables with new constants or functions. The resulting formula has the same satisfiability status as the original formula, but the two formulas need not have identical models over the same vocabulary.

9. Non-Entailed Query

Example:

[Authorised(B)]

The knowledge base proves that employee A is authorised, but it does not prove that employee B is authorised.

A prover may continue searching without finding a contradiction. First-order entailment is undecidable in general, so failure to find a proof within a step limit does not establish that the query is false.

10. Propositional versus First-Order Resolution

Feature	**Propositional**	**First-order**
Variables	No	Yes
Unification	No	Yes
Skolemisation	No	Yes
Complexity	Potentially exponential	More complex
Expressiveness	Limited	Higher

Conclusion

First-order resolution is more expressive than propositional resolution because it supports variables and relationships. However, unification and the larger search space increase computational cost.

TASK 6 – Advanced Inference Mini-Investigations (7 Marks)

1. Introduction

This task compares forward chaining and backward chaining using an organisation hierarchy.

The system contains:

- Employees.
- Managers.
- Reporting relationships.
- Department heads.

- Approval permissions.

The knowledge base contains approximately 10 definite-clause rules.

2. Knowledge Base Rules

1. Every employee reports to a manager.
2. A manager supervises employees.
3. A department head supervises managers.
4. If X reports to Y, Y is a supervisor of X.
5. If X reports to Y and Y reports to Z, Z is a senior supervisor of X.
6. A department head can approve requests.
7. A manager can approve requests from employees.
8. A senior supervisor can approve requests.
9. Employees working under an approved manager can request permission.
10. If an employee receives approval, the request is accepted.

Example facts:

[ReportsTo(Vineela,Manager1)]

[ReportsTo(Manager1,Director1)]

[DepartmentHead(Director1)]

3. Forward Chaining – Student A (M. Vineela)

Forward chaining is a data-driven inference method.

It starts with known facts and repeatedly applies rules to generate new facts until no more conclusions can be generated.

Algorithm:

1. Read the initial facts.
2. Check all rules.
3. Find rules whose conditions are satisfied.
4. Generate new facts.
5. Add new facts to the knowledge base.
6. Repeat until no new facts are generated.

4. Python Forward Chaining Program

```
facts = {
    ("ReportsTo","Vineela","Manager1"),
    ("ReportsTo","Manager1","Director1"),
    ("DepartmentHead","Director1")
}
steps = 0
while True:
    new = set()
    for f in facts:
        if f[0] == "ReportsTo":
            new.add(("Supervisor",f[2]))
            new.add(("CanRequest",f[1]))
        if f[0] == "DepartmentHead":
            new.add(("CanApprove",f[1]))
    new -= facts
    if not new: break
    facts |= new
    steps += len(new)
for f in sorted(facts): print(f)
print("Inference steps:",steps)
```



This is a simplified rule demonstration; a complete engine needs variable matching for every rule condition.

5. Backward Chaining – Student B (Arshiya Noor Mohamed)

Backward chaining is a goal-driven inference method.

It starts with a question and searches for rules and facts that can prove the answer.

Example query:

[CanApprove(Director1)?]

Steps:

1. Start with the goal.
2. Find a rule that concludes CanApprove.
3. Check its conditions.
4. Search the knowledge base for matching facts.
5. If all conditions are true, the goal succeeds.

6. Python Backward Chaining Program

```
facts = {("DepartmentHead","Director1")}
rules = {"CanApprove":"DepartmentHead"}
def backward(goal):
    if goal in facts: return True
    return (rules.get(goal[0]), goal[1]) in facts
q = ("CanApprove","Director1")
print("Query:", q)
print("Result:", backward(q))
```

```
</> Python

Run started
Initializing environment
Installing packages
Running code
Query: ('CanApprove', 'Director1')
Result: True
Run completed in 2,795.900ms
```

7. Forward versus Backward Chaining

Feature	Forward Chaining	Backward Chaining
Direction	Facts to conclusions	Goal to facts
Approach	Data-driven	Goal-driven
Starting point	Initial facts	Query
Processing	Applies available rules	Selects relevant rules
Best use	Many conclusions needed	Specific queries

8. Left Recursion and Non-Termination

Consider the following rules:

[Ancestor(x,y)leftarrow Ancestor(x,z)land Parent(z,y)]

[Ancestor(x,y)leftarrow Parent(x,y)]

The recursive rule is written first. A naive backward-chaining engine may repeatedly expand the same goal without reaching the base rule.

Remedy:

- Loop checking.
- Depth limit.
- Memoisation.

Loop checking detects repeated goals and prevents infinite recursion.

9. Experimental Comparison

The following table shows the expected qualitative behaviour. The counts must be replaced with actual program output after execution.

Number of facts	Forward chaining	Backward chaining
100	Small processing cost	Small processing cost
1,000	More facts processed	Searches relevant facts
10,000	Potentially high processing	Can remain efficient with indexing

Metrics to record:

Metric	Forward chaining	Backward chaining

Facts inferred	Record all new facts	Usually only required facts
Rule firings	Count all applicable rules	Count rules used for the goal
Iterations	Until fixed point	Until success or failure
Unifications	Rule matching	Goal matching
Repeated subgoals	Less relevant	Important to track

10. Verification Questions

Q1. Why is the choice of inference direction central to performance?

Forward chaining processes facts to generate conclusions, while backward chaining works only toward a requested goal. Therefore, the best method depends on whether the system needs many conclusions or only a few specific answers.

Q2. Why can backward chaining remain efficient with many irrelevant facts?

Backward chaining searches for facts that support the requested goal. With proper indexing, it can ignore unrelated facts and avoid unnecessary processing.

Conclusion

Forward chaining is useful when many conclusions must be generated. Backward chaining is useful when users ask specific questions. Choosing the correct direction improves performance.

TASK 7 – Design a Knowledge-Based Reasoning Component from First Principles (10 Marks)

Problem: Hospital Medication-Safety Advisor

1. Introduction

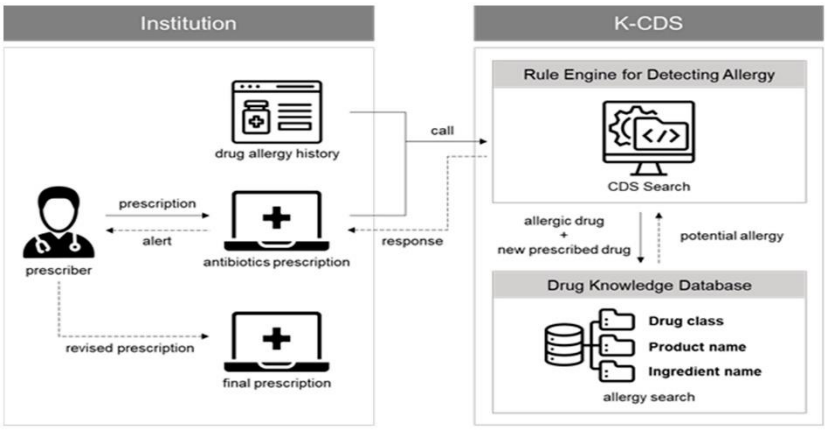
The Hospital Medication-Safety Advisor is an intelligent logical agent that checks whether a prescribed medicine is safe for a patient.

It considers:

- Patient age.
- Allergies.
- Current medicines.
- Existing medical conditions.
- Prescribed medicine.
- Drug interactions.

- Dosage restrictions.

The system provides a decision and explains the reason.



Stage 1 – Independent Design Proposal (1 Mark)

Student A – M. Vineela

Proposed Design

- Representation: First-Order Logic.
- Vocabulary: Patient, Drug, Allergy, Medicine, Condition, Prescription, Safe.
- Inference: Backward chaining.
- Explanation: Display the rules and facts responsible for the decision.

Reason: First-order logic can represent general rules for many patients and medicines without repeating the same rules.

Student B – Arshiya Noor Mohamed

Proposed Design

- Representation: First-Order Logic.
- Vocabulary: PatientID, DrugID, Age, Allergy, Interaction, Dosage, SafePrescription.
- Inference: Forward chaining.
- Explanation: Store the inference steps and display detected risks.

Reason: Forward chaining can process patient records and identify multiple medication risks.

Stage 2 – Compare the Two Designs (1 Mark)

Feature	**Student A**	**Student B**
Representation	FOL	FOL

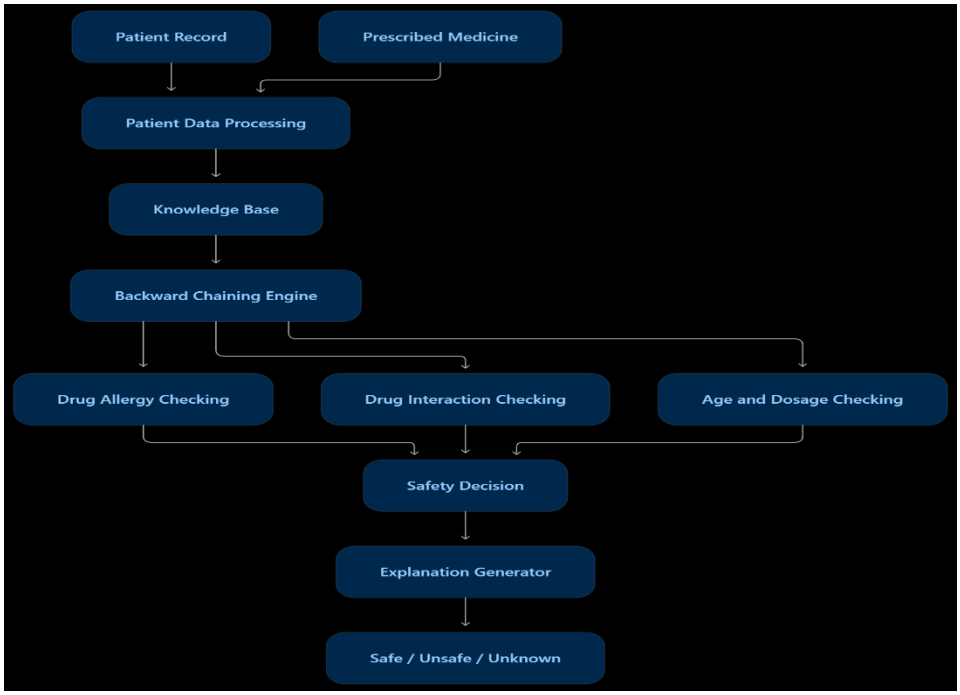
Inference	Backward chaining	Forward chaining
Focus	Specific medicine query	Multiple safety checks
Advantage	Avoids irrelevant rules	Generates multiple warnings
Limitation	Repeated goal searches	May generate unnecessary facts

Final Decision: We select First-Order Logic with backward chaining as the primary inference method because the system usually needs to answer a specific question: Is this prescription safe for this patient?

Forward chaining can also be used to precompute important warnings when patient records are updated.

Stage 3 – Joint Final Design (3 Marks)

1. System Architecture



2. Vocabulary

Predicate	**Meaning**
Patient(x)	x is a patient
Drug(y)	y is a medicine
HasAllergy(x,y)	Patient has allergy to drug
Prescribed(x,y)	Drug is prescribed

Interacts(y,z)	Two drugs interact
Taking(x,z)	Patient takes drug z
Age(x,a)	Patient age
Contraindicated(x,y)	Drug is unsuitable
Safe(x,y)	Prescription is safe
HighRisk(x,y)	Prescription has a serious risk

3. Knowledge Base Rules

1. A patient allergic to a drug must not receive that drug.
2. A drug interacting with a current medicine requires a safety warning.
3. Medicines contraindicated for a patient's condition must not be prescribed.
4. Age restrictions must be checked.
5. Dosage must be within the recommended limit.
6. A known dangerous drug interaction makes a prescription unsafe.
7. Missing patient information must produce an unknown result.
8. A prescription is safe only if all required safety checks pass.
9. A contraindicated drug cannot be considered safe.
10. A patient with no known risk and complete required information can be marked provisionally safe.

Example rules:

[forall xforall y; [Patient(x)land Drug(y)land HasAllergy(x,y)] rightarrow HighRisk(x,y)]

[forall xforall yforall z; [Taking(x,z)land Prescribed(x,y)land Interacts(y,z)] rightarrow HighRisk(x,y)]

[forall xforall y; HighRisk(x,y)rightarrowneg Safe(x,y)]

4. Complete Pseudocode

START

INPUT patient details and medicine

LOAD knowledge base

CHECK allergies, drug interactions,

contraindications and dosage

```
IF any danger exists
    DISPLAY "Prescription Unsafe"
ELSE IF information is missing
    DISPLAY "Result Unknown"
ELSE
    DISPLAY "Provisionally Safe"
DISPLAY decision, explanation and rules
END
```

5. Indexing and Agenda Strategy

For 2,000 rules and 50,000 patients, we use:

- Patient ID indexing.
- Drug ID indexing.
- Allergy-based indexing.
- Interaction lookup tables.
- Goal-directed rule selection.
- Caching of repeated queries.

These methods reduce unnecessary searches.

Stage 4 – Divide the Validation Responsibilities (3 Marks)

Student – Theoretical Validation (Arshiya Noor Mohamed)

1. Soundness

The inference system is sound if every conclusion it derives follows logically from the knowledge base.

For example:

If a patient is allergic to Drug A and the knowledge base states that allergic drugs are unsafe, the system can conclude that Drug A is unsafe for that patient.

2. Completeness

The inference system is complete if it can derive every conclusion logically supported by the knowledge base.

3. Complexity

The cost of a query depends on:

- Number of rules.
- Number of relevant facts.
- Number of unification operations.
- Number of possible inference paths.
- Indexing efficiency.

General first-order inference can be computationally expensive and is undecidable in unrestricted form.

4. Definite Clauses

Definite clauses contain exactly one positive literal.

Example:

[Allergic(x,y)rightarrow Unsafe(x,y)]

Using definite clauses makes forward and backward chaining simpler and more predictable.

However, restricting the knowledge base to definite clauses limits the types of logical relationships that can be expressed directly.

Student A – Experimental Validation (M.Vineela)

1. Thirty Test Cases

The following test cases should be used to validate the system.

No.	Test condition	Expected result
1	No allergy, no interaction, valid dose	Provisionally safe
2	Allergy to prescribed drug	Unsafe
3	Dangerous drug interaction	Unsafe
4	Invalid dosage	Unsafe
5	Age restriction violated	Unsafe
6	Contraindicated condition	Unsafe
7	Missing allergy information	Unknown
8	Missing age	Unknown
9	Missing current medicine list	Unknown

10	Two interacting medicines	Unsafe
11	Known allergy to unrelated drug	Check other conditions
12	Valid dosage and no known risks	Provisionally safe
13	Severe allergy combination	Unsafe
14	Duplicate medicine	Requires review
15	Unknown drug	Unknown
16	Medicine unsuitable for condition	Unsafe
17	Safe medicine for adult	Provisionally safe
18	Medicine below minimum age	Unsafe
19	Medicine above maximum age	Unsafe
20	Conflicting patient records	Unknown
21	Multiple current medicines	Check interactions
22	No patient record	Unknown
23	No prescription	Unknown
24	Missing dosage	Unknown
25	Valid dosage with allergy	Unsafe
26	Valid dosage with interaction	Unsafe
27	Two contraindications	Unsafe
28	Incomplete medical history	Unknown
29	All required checks passed	Provisionally safe
30	Multiple dangerous conditions	Unsafe

These are test specifications, not actual execution results.

2. Knowledge Base Size Testing

Test	**Number of rules**	**Number of patients**
Small	40	50

Medium	400	5,000
Large	2,000	50,000

3. Performance Metrics

The following metrics should be measured during implementation.

Metric	**Description**
Query latency	Time taken to answer a query
Inference steps	Number of logical operations
Memory usage	Memory consumed by the system
Explanation length	Number of reasoning steps displayed
Rule matches	Number of matching rules

4. Baseline Comparison

Compare the knowledge-based system with a simple sequential rule-checking system.

Feature	**Sequential checking**	**Indexed logical agent**
Rule search	Checks rules sequentially	Searches relevant rules
Repeated queries	Repeats checks	Can use caching
Explanation	Basic warnings	Logical proof trace
Large KB	More unnecessary searches	More focused searches

Actual performance values should be recorded after running both systems.

Stage 5 – Cross-Defence (1 Mark)

Question 1 – Student A explains experimental evidence

The experimental validation compares different knowledge base sizes and records query time, inference steps, memory and explanation length. It helps us determine whether the system performs efficiently as the number of rules and patients increases.

Question 2 – Student B explains theoretical complexity

Backward chaining is suitable because it starts from a specific query and searches only relevant rules. However, unrestricted first-order logic can have an infinite search space. Therefore, indexing, loop checking and definite-clause restrictions are important.

Stage 6 – Joint Evaluation (1 Mark)

1. Why is this problem suitable for a logical agent?

The problem contains clear medical rules and relationships between patients, medicines, allergies and conditions. A logical agent can apply these rules to produce decisions with explanations.

2. What are the limitations?

- Incomplete patient information.
- Uncertain medical data.
- Unknown drug interactions.
- Incorrect knowledge base rules.
- New medicines and changing medical guidelines.

3. What should an AI designer check before choosing a knowledge-based approach?

1. Availability of correct domain knowledge.
2. Complexity of logical rules.
3. Number of patients and medicines.
4. Required response time.
5. Ability to explain decisions.
6. Handling of missing and uncertain information.
7. Safety and expert validation.

Important safety limitation: This is an academic decision-support design. It must not independently authorize medication use. Real clinical deployment requires validated medical knowledge, clinician review and appropriate uncertainty handling.

Task 7 Conclusion

The Hospital Medication-Safety Advisor uses first-order logic and backward chaining to check prescriptions. It identifies possible risks and explains its decisions. Indexing and caching help the system handle large knowledge bases efficiently.

FINAL CONTRIBUTION RECORD

Pair Members:

- Student A – M. Vineela
- Student B – Arshiya Noor Mohamed

The following table reflects the proposed primary responsibilities. Both students must participate in every task.

Task	Student A – M. Vineela	Student B – Arshiya Noor Mohamed	Joint Work
Task 1	Symbol derivation, checker implementation	Experimental validation, soundness	Wumpus KB and inference rules
Task 2	Resolution and CNF conversion	Model-checking comparison	Smart-building policy
Task 3	FOL vocabulary and axioms	Query testing and defect analysis	Exam eligibility system
Task 4	Ground-clause experiments	Unification and MGU derivations	Lifted inference comparison
Task 5	FOL conversion and proof tree	Strategy and performance comparison	Resolution prover
Task 6	Forward chaining	Backward chaining	Organisation hierarchy
Task 7	Theoretical validation	Experimental validation	Hospital medication advisor

Overall Assignment Conclusion

This assignment demonstrates how artificial intelligence uses logical agents and knowledge-based reasoning to solve real-world problems.

We studied propositional logic, first-order logic, resolution, unification, forward chaining and backward chaining.

We also compared different inference techniques based on their efficiency, complexity and ability to explain conclusions.

Through the seven tasks, we understood that selecting the correct knowledge representation and inference method is important for developing an efficient intelligent system.